

want to create a copy of the Superclass, we need to somehow, fire off the Superclass object's Constructor.

Whenever you extended another class, (a Superclass) you have created an instance of every class above your class in the class hierarchy. When the Constructor for Sales fires, it silently also fires the Constructors for JApplet and every other class above JApplet in the hierarchy.

```
public class Sales extends JApplet
{
    ...
}
```

When the Constructor for Sales fires, it silently also fires the Constructors for JApplet and every other class above JApplet in the hierarchy.

The Subclass adds to what it inherits from the Superclass, because we say it adds, that implies it already has everything else. By the time your Subclass is created, it already has an instance of every Superclass above it and every single thing the Superclasses contain. Every method, every data variable present in the Superclass is also present in the Subclass. The point to note here is that if your class has everything above it then you can say it is an example of the classes above it. Finally, since the Subclass has everything the Superclass has, we can say that every Subclass is also an object of the Superclass.

So, let's ponder this: JOptionPane inherits from JComponent. The JOptionPane inherits everything JComponent possesses. So, at the same time we can say JOptionPane is a JComponent, JOptionPane is a Container, JOptionPane is a Component and JOptionPane is an Object.

Classes - Object, Component and Container are Indirect Superclasses to the JOptionPane. JComponent is the Direct Superclass to the JOptionPane. We say JOptionPane is a JComponent, a Container, Component, and an Object.

5.2 Impact of access modifiers

Access Modifiers are among the thorniest and most confusing aspects of OOP. But, remember, encapsulation is one of the primary benefits of object orientation, so access is important. The familiar private access modifier lets us shield our instance variables from the prying eyes of the outside